

Route planning algorithms evaluation

1. README / Code Execution

To evaluate and compare the average number of iterations for the implemented algorithms, run the `eval_algorithms.py` script. Please keep in mind that:

- It outputs the **average** number of iterations for each algorithm and for both cities (with the relative number of nodes and edges of the relative graph), calculating them **on the exact same pairs of locations** (start and destination).
- Furthermore, it shows the cost of the last path found in terms of length (meters) and plots a visual comparison of the final routes on the map using 4 different colors. If you see just one color, it means that the paths perfectly overlap.
- The console will **only print the averages** without giving you any information about the number of steps for each single run (the nth out of 10). If you want to see them printed, uncomment the section going from line 127 to line 130 inside the code. This will display a table with the number of iterations for every algorithm at each of the 10 calculations.

Note: The standalone scripts `Astar.py` and `Dijkstra.py` are included as requested and can be run independently. However, `eval_algorithms.py` is the main script designed to evaluate all the algorithms' performances on the exact same set of random points. Additionally, console outputs, plotting, and other aesthetic factors are ignored in the standalone files.

Results

The tables below report the number of iterations required by each algorithm to find the destination node across 10 randomly selected pairs in two different cities.

AOSTA [Nodes:825, Edges: 1723]				
#iter	Dijkstra	A* Manhattan	A* Euclidean	A* Haversine
1	33	17	16	16
2	819	747	808	815
3	804	676	737	764
4	50	31	31	35

AOSTA [Nodes:825, Edges: 1723]				
5	567	41	41	43
6	819	707	764	815
7	184	20	32	35
8	36	24	24	22
9	42	15	16	19
10	293	33	28	101
AVG	364.7	231.1	249.7	266.5

TURIN [Nodes:11768, Edges: 25206]				
#iter	Dijkstra	A* Manhattan	A* Euclidean	A* Haversine
1	6559	223	283	234
2	9965	107	670	1512
3	1449	131	249	245
4	5438	71	233	417
5	4450	62	173	402
6	10449	221	243	1412
7	7861	191	117	758
8	390	34	61	72
9	5837	636	1046	1132
10	6282	278	619	666
AVG (/10)	5868.0	195.4	369.4	685.0

3. Performance Analysis

The A* algorithm significantly outperforms Dijkstra's algorithm; however, the factor of performance improvement is directly related to the size of the graph:

- **Using the Aosta graph:** The difference in iterations is modest, with A* using the Haversine heuristic being roughly **1.5 times faster** than Dijkstra.
- **Using the Turin graph:** The efficiency improvement scales significantly. A* with Haversine is about **8.5 times faster** than Dijkstra, while A* with the Manhattan heuristic can be up to **20 times faster**.

We must always keep in mind the trade-off between optimality and speed. As we will discuss in the next section, the Manhattan heuristic often fails to find the same optimal path as Dijkstra. In fact, while **Haversine** requires the highest number of iterations among the A* variants, it is the only one guaranteed to **always find the exact same optimal path as Dijkstra** (with rare exceptions for extremely short micro-distances due to floating-point approximations). Manhattan and Euclidean perform fewer iterations but behave in a greedy manner, often returning sub-optimal paths. The underlying reasons for this behavior are explained in the following section.

4. Methodology and Considerations

To ensure mathematical consistency and allow the algorithms to function properly, specific adaptations had to be made to the original codebase. These methodological choices and their impact on the algorithms' performances are detailed below.

Unit Consistency and Heuristic unit choice

In OpenStreetMap, the edge length is provided in **meters**, while the maximum speed (**maxspeed**) is expressed in **km/h**. To prevent unit mismatch errors the speed was converted to m/s (by dividing by 3.6). For the same reason, the **heuristic function** $h(n)$ (which naturally calculates a geographic distance in meters) also had to be **divided by a speed factor** to return a time estimate in seconds. This fundamental step ensures that the A* evaluation function $f(n) = g(n) + h(n)$ computes homogeneous units.

The Longitude Correction Issue

The assignment formulas for Manhattan and Euclidean distances lack the **cos(phi) correction factor** for longitude (as explained in this website, applying this term is fundamental to avoid spatial deformations caused by the fact that the physical distance between meridians shrinks as we move closer to the poles [How to measure distances between two coordinates: A data analyst's guide | by Mochamad Kautzar Ichramsyah | CodeX | Medium](#)). A degree of longitude in Northern Italy (Turin/Aosta) is significantly shorter than a degree at the equator, making the constant conversion factor of 111,320 meters per degree inaccurate for this specific region of the Earth. This breaks the **admissibility** rule of A* ($h(n)$ must never **overestimate** the true cost). As a result, A* becomes extremely greedy and it closes the search quickly, often settling for sub-optimal routes.

Heuristic Tightness and the Speed Factor (40 vs. 130 km/h)

To achieve unit consistency with the actual path cost $g(n)$ (which is evaluated in seconds), the spatial distance calculated by the heuristic $h(n)$ had to be divided by a reference velocity. Two main candidates were considered for this conversion: **130 km/h** (the absolute maximum possible speed on the road network) and **40 km/h** (the average urban speed limit observed within the graph).

Testing revealed that dividing by **130 km/h** makes the $h(n)$ value so small that A* loses its directional guidance, severely slowing down the search and making it behave similarly to Dijkstra's algorithm. Although Manhattan and Euclidean heuristics were significantly more accurate, with this speed value this approach completely lacks the rapidity advantages that makes A* more suitable than Dijkstra for some applications (i.e. real time fast route search).

(Note: If the goal is to guarantee that **every heuristic leads to the exact optimal path**, ignoring speed improves, it is sufficient to change the `MAX_SPEED_MS` value back to 130 inside the `calculate_h()` function inside the [Astar.py](#) code at line 71)

Conversely, choosing **40 km/h** establishes a high **Heuristic Tightness**, leading to two distinct practical behaviors:

- **Haversine:** By dividing the straight-line distance by the average urban speed, the time estimation becomes incredibly tight while remaining admissible. This perfectly **balances optimality and speed**: it reliably finds the exact optimal path while drastically (most of the times) reducing the search space (up to an 8.5x reduction in iterations compared to Dijkstra in Turin).
- **Manhattan & Euclidean:** Since these formulas already overestimate the geographic distance (because of the $\cos(\phi)$ omission), dividing them by a tighter speed bound amplifies the error. They break admissibility and cause the algorithm to effectively behave as a Weighted A*, rushing greedily toward the target to minimize computation time at the expense of path optimality.

In conclusion, Dijkstra provides the absolute ground truth for the shortest path but is computationally expensive. A* with Haversine at an average speed divisor offers the best trade-off, maintaining perfect optimality while massively reducing the search space on large networks.